From Numbers to Vectors and Matrices

Recap of lecture 1 : we understand how floating point arithmetic generally works (play with formats on Float Toy). **But** individual numbers aren't enough : real problems involve millions to billions of numbers arranged in specific ways. Today : the most common arrangement, *vectors & matrices*.

Definition

SCALARS, VECTORS, MATRICES, TENSORS
Key numerical data structures organized by dimension : scalar, row vector, column vector, matrix, tensor. All are **nd-arrays** (general n-dimensional storage); vectors and matrices are representations of *linear functions*.

Definition

LINEAR FUNCTION
f is a linear function *if and only if*

$$f(\mathbf{x}+\mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y}) \qquad \text{(additivity)}$$
$$f(\alpha\mathbf{x}) = \alpha f(\mathbf{x}) \quad (\alpha \in \mathbb{R}) \qquad \text{(homogeneity)}$$

In this case $f(\mathbf{x}) = \mathbf{A}\mathbf{x}$ fully encodes everything that f does!

Key formulas

Matrix-vector multiplication

$$(\mathbf{A}\mathbf{x})_i = \sum_{k=1}^{n} A_{ik} x_k$$

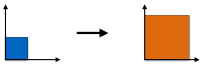
With the columns $\mathbf{a}^{(1)}, \mathbf{a}^{(2)}, \mathbf{a}^{(3)}$ of $\mathbf{A}$:

$$\mathbf{A}\mathbf{x} = \mathbf{a}^{(1)} x_1 + \mathbf{a}^{(2)} x_2 + \mathbf{a}^{(3)} x_3$$

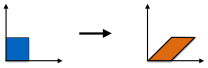
a linear combination of the columns (if they are linearly dependent, they span e.g. only a 2D subspace).

Basic matrix transformations in 2D

Scaling:
$$\begin{bmatrix} \mathbf{v}'_x \\ \mathbf{v}'_y \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} \mathbf{v}_x \\ \mathbf{v}_y \end{bmatrix}$$



Shear:
$$\begin{bmatrix} \mathbf{v}'_x \\ \mathbf{v}'_y \end{bmatrix} = \begin{bmatrix} 1 & c \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{v}_x \\ \mathbf{v}_y \end{bmatrix}$$



Rotation:
$$\begin{bmatrix} \mathbf{v}'_x \\ \mathbf{v}'_y \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} \mathbf{v}_x \\ \mathbf{v}_y \end{bmatrix}$$



Mirror:
$$\begin{bmatrix} \mathbf{v}'_x \\ \mathbf{v}'_y \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{v}_x \\ \mathbf{v}_y \end{bmatrix}$$



Key idea

Why care about linear functions?

- Solving linear problems is one of the (few) numerical problems that we know to solve **really well** : when you can turn something into a linear system, good job, you are done. *Nonlinear* methods are *fragile*.
- Almost everything boils down to a linear system at some point.

Course topics ("veggies first, then the cake") : 1. floating point arithmetic & error; 2. linear systems, LU (*today*); 3. conditioning & least squares basics; 4. least squares, QR; 5. writing efficient numerical code; 6. eigenanalysis, SVD; 7. nonlinear problems; 8. automatic differentiation; 9. neural networks.

Cost of Matrix Operations

Key formulas

Matrix-vector multiplication : $\mathcal{O}(n^2)$
$\mathbf{b} \in \mathbb{R}^{10000}$, $\mathbf{A} \in \mathbb{R}^{10000\times 10000}$ (100 million entries), measured on an M1 Mac :

Code	Time	
`b = A @ x`	11 ms	
rows : `b[i] = A[i, :] @ x`	66 ms	∼6×
cols : `b += A[:, j] * x[j]`	309 ms	∼28×
2 loops : `b[i] += A[i, j] * x[j]`	30'282 ms	∼2'752×

Definition

MATRIX MULTIPLICATION

$$(\mathbf{A}\mathbf{B})_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

Why this definition? If $f(\mathbf{x}) \leftrightarrow \mathbf{A}\mathbf{x}$ and $g(\mathbf{x}) \leftrightarrow \mathbf{B}\mathbf{x}$, then the composition $g(f(\mathbf{x})) \leftrightarrow \mathbf{B}\mathbf{A}\mathbf{x} = \mathbf{C}\mathbf{x}$.

Key formulas

Matrix-matrix multiplication : $\mathcal{O}(n^3)$
$\mathbf{A}, \mathbf{B} \in \mathbb{R}^{1000\times 1000}$ (1 million entries) :

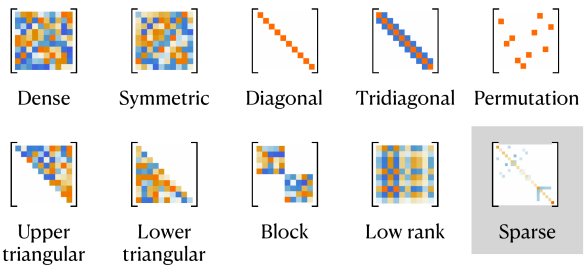
Code	Time	
`C = A @ B`	19 ms	
loop : `C[:, j] = A @ B[:, j]`	236 ms	∼6×
triple loop	341'495 ms	∼17'973×

Key idea

Is $\mathcal{O}(n^3)$ really true? Volker Strassen, 1969 : $O(n^{2.8074})$. Virginia Williams et al., 2023 : $O(n^{2.371552})$. Still an active area of research : quite possible that the true cost is $O(n^2)$, *nobody knows!* Many works are *galactic* algorithms; in practice, the naïve algorithm tends to be fastest.

Matrix Zoo and Sparse Matrices

Matrix zoo : dense, symmetric, diagonal, tridiagonal, permutation, upper triangular, lower triangular, block, low rank, sparse. Also important : positive definite & orthogonal matrices.



Definition

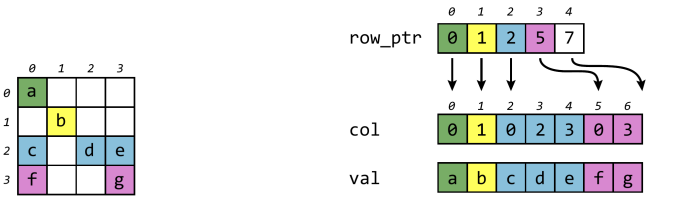
SPARSE MATRIX
Almost all ($> 99\%$) entries are zero.

- Useful : can avoid storing the zeros & skip them during computation.
- Almost every matrix algorithm has an analogous *sparse* version (or several !).
- Encodings : *Compressed Sparse Row* (CSR) or *Compressed Sparse Column* (CSC) (`scipy.sparse.csr_matrix` & `scipy.sparse.csc_matrix`).

Example

- Protein modeling, social networks, triangle meshes of 3D objects : sparsity arises because most things *aren't* connected.
- Discretizing continuous equations : 1D heat equation $\frac{\partial^2 f(x)}{\partial x^2} = 0$ becomes $-f(x_{i-1}) + 2f(x_i) - f(x_{i+1}) = 0$, one row $(0\,0\,0\,-1\,2\,-1\,0\,0\,0)$ of a HUGE matrix. Intuition : heat wants to spread out and become uniform, can't have a hotspot.

Sparse matrix-vector multiplication (CSR)



Original matrix Compressed matrix (CSR for

Key formulas

Row i uses entries `row_ptr[i]` to `row_ptr[i+1]` of `col` and `val` :

```
y = np.zeros(n)
for i in range(n):
  for k in range(A.rowptr[i], A.rowptr[i+1]):
    y[i] += A.val[k] * x[A.col[k]]
```

Linear Systems

Today : dense, upper triangular, lower triangular and permutation matrices.

Definition

LINEAR SYSTEM

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \qquad \mathbf{A} \in \mathbb{R}^{m \times n},\ \mathbf{x} \in \mathbb{R}^n,\ \mathbf{b} \in \mathbb{R}^m$$

Can $\mathbf{b}$ be expressed as a linear combination of the columns of $\mathbf{A}$? The solution may not exist or may not be unique! This lecture : the *square* case ($m = n$).

Key idea

Geometric interpretation

- **Column-based view** : try to reach point $\mathbf{b}$ using a linear combination of basis vectors $\mathbf{a}^{(i)}$ (columns). $\mathbf{x}$ tells us how much of each axis we need.
- **Row-based view** : compute the intersection of planes with normals $\mathbf{a}^{(i)}$ (rows) and offset b_i : $\mathbf{a}^{(i)} \cdot \mathbf{x} = b_i$.

Warning

Pure versus numerical mathematics : concepts like invertibility become more nuanced.

$$\begin{pmatrix} 1 & 0 \\ 1 & \epsilon \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

Conceptually the matrix is nonsingular for every $\epsilon \neq 0$ and singular for $\epsilon = 0$; in IEEE754 FP it becomes gradually "more singular" from $\epsilon = 10^{-1}$ to 10^{-10}, 10^{-50}, 0.

Gaussian Elimination

Definition

GAUSSIAN ELIMINATION (MATH-111 style)
On the augmented matrix $[\,\mathbf{A} \mid \mathbf{b}\,]$, three ingredients :

- **permute rows**;
- **scale rows**;
- **add a (scaled) version of a row** to another.

Example

Add $-5\times$ row 1 to row 2 :

$$\left[\begin{array}{ccc|c} 1 & 0 & 3 & 1 \\ 5 & 1 & 1 & 2 \\ 4 & 2 & 0 & 3 \end{array}\right] \rightarrow \left[\begin{array}{ccc|c} 1 & 0 & 3 & 1 \\ 0 & 1 & -14 & -3 \\ 4 & 2 & 0 & 3 \end{array}\right]$$

Key idea

Each step uses a **pivot element** on the diagonal. Observation : the values of $\mathbf{b}$ are updated but don't really influence the process.

Warning

Instabilities can occur even for nonzero elements (e.g. a tiny pivot ε) : the pivot element must be chosen carefully. **Practical solution** : pick the element with the largest absolute value.

Solving Repeatedly : Don't Invert

A common situation : the same system must be solved over and over again, $\mathbf{A}\mathbf{x}^{(1)} = \mathbf{b}^{(1)}$, $\mathbf{A}\mathbf{x}^{(2)} = \mathbf{b}^{(2)}$, … Can we *precompute* something ?

Warning

Anti-pattern – don't do this : precompute $\mathbf{A}^{-1}$, then one matrix-vector multiplication per solve, $\mathbf{x}^{(i)} = \mathbf{A}^{-1}\mathbf{b}^{(i)}$.

1. Computation of the inverse can be numerically unstable.
2. Inverse + matrix multiplies require slightly more FLOPs than the alternative (LU).
3. Generally, the inverse of a sparse matrix is dense : will run out of memory.

LU Factorization

Definition

LU FACTORIZATION (our first decomposition)
Run Gaussian elimination on $\mathbf{A}$ alone (no right-hand side $\mathbf{b}$), keeping track of the transformations it performs. They can be written as matrices $\mathbf{L}$ (lower triangular) and $\mathbf{U}$ (upper triangular) :

$$\mathbf{A} = \mathbf{LU}$$

Then "apply" $\mathbf{L}$ and $\mathbf{U}$ to any right-hand side $\mathbf{b}$ very efficiently.

Key formulas

A single elimination step is a matrix :

$$\begin{bmatrix} 1 & 0 & 0 \\ c & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \mathbf{A} \text{ adds } c \times \text{row 1 to row 2}$$

Its inverse is simple :

$$\begin{bmatrix} 1 & 0 & 0 \\ c & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ -c & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

A lower triangular matrix; the product of lower-triangular matrices is always lower-triangular.

Key formulas

Putting things together : $\mathbf{U}$ is the traditional output of Gaussian elimination, $\mathbf{L}$ reconstructs $\mathbf{A}$:

$$\begin{aligned} \mathbf{U} &= \mathbf{E}_6\mathbf{E}_5\mathbf{E}_4\mathbf{E}_3\mathbf{E}_2\mathbf{E}_1\mathbf{A} \\ \mathbf{L} &= (\mathbf{E}_6\mathbf{E}_5\mathbf{E}_4\mathbf{E}_3\mathbf{E}_2\mathbf{E}_1)^{-1} \\ &= \mathbf{E}_1^{-1}\mathbf{E}_2^{-1}\mathbf{E}_3^{-1}\mathbf{E}_4^{-1}\mathbf{E}_5^{-1}\mathbf{E}_6^{-1} \end{aligned}$$

Example

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 2 & 4 \end{bmatrix}, \quad \mathbf{E}_1 = \begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix} \text{ (row 2} - = 2 \times \text{row 1)}$$

$$\mathbf{U} = \mathbf{E}_1\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}, \quad \mathbf{L} = \mathbf{E}_1^{-1} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} \rightarrow \mathbf{A} = \mathbf{LU}$$

Warning

We won't look at low-level details of how the LU factorization is computed (ditto for others).

Using an LU Decomposition

Key formulas

$$\mathbf{x} = \mathbf{U}^{-1}(\mathbf{L}^{-1}\mathbf{b})$$

Important : here, $\cdot^{-1}$ does *not* mean computing a matrix inverse, but solving triangular systems (forward substitution with $\mathbf{L}$, back substitution with $\mathbf{U}$).

Key idea

Forward/back-substitution have the same complexity as matrix-vector multiplication : solving a triangular system takes $O(n^2)$, i.e. *all the hard work is done already!*

Definition

LU WITH PIVOTING (column pivoting, also known as partial pivoting)

$$\mathbf{A} = \mathbf{PLU}, \qquad \mathbf{x} = \mathbf{U}^{-1}(\mathbf{L}^{-1}(\mathbf{P}^{-1}\mathbf{b}))$$

$\mathbf{P}$ is a **permutation matrix** : each row/column contains a single 1, the rest is zero.

Example

In SciPy

```
import scipy.linalg as la
lu, piv = la.lu_factor(A)
x = la.lu_solve((lu, piv), b)
```

Returns a pivot element vector and a single 2D array : the diagonal elements of $\mathbf{L}$ are 1, so both components can be packed tightly.

Cholesky Decomposition

Definition

CHOLESKY DECOMPOSITION (symmetric positive definite matrices)

$$\mathbf{A} = \mathbf{LL}^T$$

Almost the same as an LU decomposition, but half as expensive (storage, compute).

Key formulas

2×2 example :

$$\begin{bmatrix} A_{11} & A_{21} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix} \begin{bmatrix} L_{11} & L_{21} \\ 0 & L_{22} \end{bmatrix}$$

$$L_{11} = \sqrt{A_{11}}, \quad L_{21} = \frac{A_{21}}{L_{11}}, \quad L_{22} = \sqrt{A_{22} - L_{21}^2}$$

Weekly Summary

Summary

- Linear functions ↔ matrices; matrix product ↔ composition.
- Matrix-vector $\mathcal{O}(n^2)$, matrix-matrix $\mathcal{O}(n^3)$; vectorized code is orders of magnitude faster than Python loops.
- Sparse matrices (CSR / CSC) skip the zeros.
- Gaussian elimination with pivoting (largest absolute value).
- Never compute $\mathbf{A}^{-1}$: factor $\mathbf{A} = \mathbf{PLU}$ once ($\mathcal{O}(n^3)$), then solve triangular systems ($O(n^2)$) per right-hand side.
- SPD matrices : Cholesky $\mathbf{A} = \mathbf{LL}^T$, half the cost.